

▼ 基础环境准备

- 工具更新
- 安装工具

▼ 配置安卓开发的核心工具

- 下载并组织安卓命令行工具
- ▼ 配置 `sdkmanager` 安装核心组件
 - 临时设置环境变量
 - 接受所有许可协议
 - 将环境变量写入 `~/.bashrc` 文件

▼ 编译 MNN C++ 库

- 下载 MNN 源码
- 事故排查
- 执行编译

▼ 打包 APK

- 事故排查
- 打包命令

▪ 下载安装包

官方文档是完美的，它假设你拥有一个完美的环境。但现实是，我们往往从一台一无所有的全新服务器开始。这篇指南，就是为了填补从“一无所有”到“完美环境”之间的巨大鸿沟。

我们将完整记录在一台全新的、纯净的 Ubuntu 服务器上，通过纯命令行，踩遍所有可能遇到的坑，最终成功编译出 MNN LlmChat 应用的 APK。

最终目标：在 `.../app/build/outputs/apk/standard/debug/` 目录下，生成 `app-standard-debug.apk` 文件。

基础环境准备

工具更新

```
sudo apt update
sudo apt upgrade -y
```

安装工具

安装 C++ 编译工具 (build-essential, cmake)、源码管理工具 (git)、解压工具 (unzip) 以及 Java 开发环境 (openjdk-17-jdk)。

```
sudo apt install -y build-essential git cmake openjdk-17-jdk unzip
```

安装完成后，通过 `java --version` 验证 Java 是否就绪。

配置安卓开发的核心工具

下载并组织安卓命令行工具

从 Android Studio 官网下载 "[Command line tools only](#)" 的 Linux 版本。使用 `wget` 下载，并按照 Google 推荐的目录结构来解压和组织文件。

```
mkdir -p ~/Android/Sdk
unzip commandlinetools-linux-*.zip -d ~/Android/Sdk/
mkdir -p ~/Android/Sdk/cmdline-tools/latest
mv ~/Android/Sdk/cmdline-tools/* ~/Android/Sdk/cmdline-tools/latest/
```

配置 `sdkmanager` 安装核心组件

临时设置环境变量

```
export ANDROID_HOME=$HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/cmdline-tools/latest/bin
```

接受所有许可协议

使用 `sdkmanager` 接受所有许可协议，并安装必要的平台工具、构建工具、SDK 平台和 NDK。

```
yes | sdkmanager --licenses
sdkmanager "platform-tools" "platforms;android-34" "build-tools;34.0.0" "ndk;25.2.9519653" "cn
```

将环境变量写入 `~/.bashrc` 文件

```
vim ~/.bashrc
```

在文件末尾添加以下所有内容：

```
# Android SDK & NDK Environment Variables
export ANDROID_HOME=$HOME/Android/Sdk
export ANDROID_NDK_HOME=$ANDROID_HOME/ndk/25.2.9519653
export PATH=$PATH:$ANDROID_HOME/cmdline-tools/latest/bin
export PATH=$PATH:$ANDROID_HOME/platform-tools
export PATH=$PATH:$ANDROID_HOME/build-tools/34.0.0
export PATH=$PATH:$ANDROID_NDK_HOME
```

编译 MNN C++ 库

下载 MNN 源码

```
cd ~
git clone [https://github.com/alibaba/MNN.git](https://github.com/alibaba/MNN.git)
```

事故排查

1. build_64.sh 找不到 NDK

报错: Could not find toolchain file: /build/cmake/android.toolchain.cmake。

原因: 官方脚本设计存在缺陷。

对策: 放弃官方脚本, 直接调用 cmake。

2. 找不到主配置文件

报错: does not appear to contain CMakeLists.txt.。

原因: MNN 的主 CMakeLists.txt 在项目根目录, 而我们执行 cmake .. 时, 相对路径错误。

对策: 正确的相对路径 ../../../, 从 build_64 目录指向项目根目录。

3. CMake 版本不兼容

报错: Target "... " is an OBJECT library that may not have PRE_BUILD... commands.。

原因: 服务器上的新版 cmake (3.22) 比 MNN 项目脚本编写时使用的版本更严格, 禁止了某些不规范的使用法。

对策: 简化引擎: 在 cmake 命令中, 将所有导致此错误的非核心模组 (如视觉 -DLLM_SUPPORT_VISION=OFF、音频 -DLLM_SUPPORT_AUDIO=OFF 等) 全部禁用。

组合拳: 发现核心的 llm 模组自身也存在此问题, 必须通过修改源码来解决。编辑 ~/MNN/transformers/llm/engine/CMakeLists.txt 文件, 找到第 60 行左右的 add_custom_command(TARGET llm ...) , 在该行的最前面加上 #, 将其注释掉。

4. libMNN.so 位置错误

问题: 编译出的 libMNN.so, 被 find . -name "libMNN.so" 命令发现在 OFF/arm64-v8a / 目录里。

原因: MNN 的 CMake 脚本存在 Bug, 错误地将设定的编译选项 OFF 当成了一个文件夹名。

对策: 手动创建正确的 lib 文件夹, 并用 mv 命令将 libMNN.so 移动到 Gradle 期望的 lib/ 目录下。

执行编译

```
cd ~/MNN/project/android
rm -rf build_64 && mkdir build_64 && cd build_64

cmake ../../../../ \
-DMAKE_TOOLCHAIN_FILE=$ANDROID_NDK_HOME/build/cmake/android.toolchain.cmake \
-DANDROID_ABI=arm64-v8a \
-DANDROID_PLATFORM=android-21 \
-DMNN_BUILD_LLM=true \
-DMNN_ARM82=true \
-DMNN_OPENCL=true \
-DMNN_BUILD_OPENCV=OFF \
-DLLM_SUPPORT_VISION=OFF \
-DMNN_BUILD_AUDIO=OFF \
-DLLM_SUPPORT_AUDIO=OFF \
-DMNN_BUILD_DIFFUSION=OFF
```

编译完成后，开始 make 锻造，并手动修正。

```
make -j$(nproc)
mkdir -p lib
mv OFF/arm64-v8a/libMNN.so lib/
```

通过 `ls -l lib/` 确认 `libMNN.so` 已就位。至此，核心引擎编译完毕。

打包 APK

事故排查

1. 链接器错误

报错： `undefined symbol: MNN::DIFFUSION::...` 。

原因：之前在编译时，为了绕过 CMake 的 Bug，选择了不安装部分模组（`-DMNN_BUILD_DIFFUSION=OFF`）。但 App 的 C++ 源码上有对应接口。

对策：编辑 `~/MNN/apps/Android/MnnLlmChat/app/src/main/cpp/diffusion_jni.cpp`，和 `~/MNN/apps/Android/MnnLlmChat/app/src/main/cpp/diffusion_session.cpp`，在这两个文件的最顶端加上 `#if 0`，在最末尾加上 `#endif`，告诉编译器彻底忽略这两个文件的所有内容。

2. 依赖下载超时/失败

报错： `Read timed out` 或 `401 Unauthorized` 。

原因：服务器的网络环境访问国外的 `jitpack.io` 或国内需要授权的镜像仓库时，连接缓慢或被拒绝。

对策：修改 `settings.gradle` 文件，采用多源策略，优先使用腾讯云的公共镜像。

编辑 `~/MNN/apps/Android/MnnLlmChat/settings.gradle`，找到 `dependencyResolutionManagement` 代码块，用下面的内容替换掉其中的 `repositories { ... }` 部分：

```
repositories {  
    maven { url ' [https://mirrors.tencent.com/nexus/repository/maven-public/](https://mirrors.  
    google()  
    mavenCentral()  
    maven { url ' [https://jitpack.io](https://jitpack.io)' }  
}
```

3. 内存不足

报

错:

Gradle build daemon disappeared unexpectedly (it may have been killed or may have crashed)
。

原因: Gradle 编译极其消耗内存, 导致服务器的 Linux 系统启动了 OOM Killer (Out Of Memory Killer) 机制, 杀死了最耗内存的 Gradle 进程。

对策: 编辑 `gradle.properties` 文件, 限制 Gradle 的最大内存为 1024MB

(1GB): `org.gradle.jvmargs=-Xmx1024m` 。

打包命令

回到 App 根目录, 执行最终的打包命令。

```
cd ~/MNN/apps/Android/MnnLlmChat  
./gradlew assembleDebug
```

当你看到终端输出绿色的 BUILD SUCCESSFUL 字样时, 恭喜你, 你已经征服了这座高山! `app-standard-debug.apk` ——正躺在以下目

录: `~/MNN/apps/Android/MnnLlmChat/app/build/outputs/apk/standard/debug/`

下载安装包

在自己的电脑终端, 使用 `scp` 命令下载安转包到本地。

```
scp root@[IP]:~/MNN/apps/Android/MnnLlmChat/app/build/outputs/apk/standard/debug/app-standard-
```

至此, 整个旅程圆满结束!